

Mathematical Explanation of Bounding Box Computation in 2D Gaussian Splatting

1 Mathematical Foundation

The bounding box computation derives an axis-aligned bounding box (AABB) for each projected 2D Gaussian splat in screen space by calculating the projected center (means2d) and half-extents (extents) of the ellipse resulting from the perspective projection of the local unit circle, representing the Gaussian disk in tangent space. The approach leverages projective geometry to compute these values efficiently without explicit matrix inversion, ensuring numerical stability and computational efficiency.

The local 2D Gaussian is modeled as a unit circle in the tangent plane, defined by the conic equation $h_l^T Q h_l = 0$, where $h_l = [u, v, 1]^T$ is the homogeneous coordinate in local space, and $Q = \text{diag}(1, 1, -1)$ represents the quadric $x^2 + y^2 - 1 = 0$. The projection to screen space uses the transformation matrix T_{sl} , yielding the projected conic:

$$C_{\text{screen}} = T_{sl}^{-T} Q T_{sl}^{-1}.$$

To avoid inverting T_{sl} , the computation works with the inverse conic $S = C_{\text{screen}}^{-1} = T_{sl} Q^{-1} T_{sl}^T$, where $Q^{-1} = Q$ due to its diagonal form. The center and extents of the projected ellipse are extracted using polar duality:

- **Projected Center:** The center means2d = $[u_0, v_0]$, where:

$$u_0 = \frac{S_{02}}{S_{22}}, \quad v_0 = \frac{S_{12}}{S_{22}}.$$

- **Half-Extents:** The extents extents = $[\Delta u, \Delta v]$, where:

$$\Delta u = \sqrt{u_0^2 - \frac{S_{00}}{S_{22}}}, \quad \Delta v = \sqrt{v_0^2 - \frac{S_{11}}{S_{22}}}.$$

These formulas derive from the tangency conditions for axis-parallel lines to the ellipse. For vertical lines (constant u), the dual condition $l^T S l = 0$ with $l = [1, 0, -u]^T$ yields a quadratic equation whose roots define the minimum and maximum u , with the half-width Δu as shown. The same applies for v . This ensures an exact AABB for the projected ellipse.

Given $M = T_{sl}^T$, the inverse conic becomes:

$$S = M Q^{-1} M^T,$$

where M aligns with the paper's notation $M = (WH)^T$, with W as the world-to-screen transformation and H encoding the Gaussian's geometry. The conic elements are computed as: - $S_{22} = e_3^T S e_3 = (M^T e_3)^T Q^{-1} (M^T e_3)$, using the third row of M . - Similar dot products yield $S_{00}, S_{11}, S_{02}, S_{12}$.

The final radius is computed as the ceiling of three times the maximum half-extent, providing a conservative bound suitable for tile-based rasterization and culling, accounting for the Gaussian falloff (approximately 3σ coverage for $\exp(-(u^2 + v^2)/2)$).

1.1 Derivation of Projected Center and Half-Extents

The derivation assumes S is a symmetric 3×3 matrix:

$$S = \begin{pmatrix} S_{00} & S_{01} & S_{02} \\ S_{01} & S_{11} & S_{12} \\ S_{02} & S_{12} & S_{22} \end{pmatrix},$$

with $S_{22} > 0$ for a valid (bounded) ellipse.

1.1.1 For the u -Direction (Δu)

Consider vertical tangent lines $u = \text{constant}$, represented as $l = [1, 0, -u]^T$.

Substitute into $l^T S l = 0$:

$$S_{00} - 2S_{02}u + S_{22}u^2 = 0.$$

The quadratic is:

$$S_{22}u^2 - 2S_{02}u + S_{00} = 0.$$

Roots:

$$u = \frac{S_{02} \pm \sqrt{S_{02}^2 - S_{22}S_{00}}}{S_{22}}.$$

Center:

$$u_0 = \frac{S_{02}}{S_{22}}.$$

Half-extent:

$$\Delta u = \sqrt{u_0^2 - \frac{S_{00}}{S_{22}}}.$$

1.1.2 For the v -Direction (Δv)

For horizontal lines $l = [0, 1, -v]^T$:

$$S_{11} - 2S_{12}v + S_{22}v^2 = 0.$$

Center:

$$v_0 = \frac{S_{12}}{S_{22}}.$$

Half-extent:

$$\Delta v = \sqrt{v_0^2 - \frac{S_{11}}{S_{22}}}.$$

2 Code Breakdown with Mathematical Integration

The code approximates the perspective projection using an affine transformation $T_{sl} = K @ T_{cl}$, sufficient for bounding. Below is a step-by-step breakdown:

1. Transformation Matrix Computation:

- Compute RS_{wl} , the rotation-scaling matrix in world space.
- Transform to camera space: $RS_{cl} = R_{cw} @ RS_{wl}$.
- Assemble $T_{cl} = [RS_{cl}[:, :, : 2], \text{means}_c]$.
- Apply intrinsics: $T_{sl} = K @ T_{cl}$.
- Set $M = T_{sl}^T$.

2. Normal Computation:

- Extract normals from RS_{cl} 's third column and adjust sign for front-facing.

3. Conic Element Computation:

- $\text{test} = [1, 1, -1]$ for $\text{diag}(Q)$.
- Compute $d = \sum(M[:, 2] \cdot M[:, 2] \cdot \text{test}) \approx S_{22}$.
- If $|d| > \epsilon$, set $f = \text{test}/d$; else zero.

4. Projected Center (means2d):

- $\text{means2d} = \sum(M[:, 2, :] \cdot M[2 : 3, :] \cdot f)$, equivalent to $[S_{02}/S_{22}, S_{12}/S_{22}]$.

5. Half-Extents (extents):

- Compute $\text{temp} = \sum(M[:, 2, :]^2 \cdot f)$ for $S_{00}/S_{22}, S_{11}/S_{22}$.
- $\text{extents} = \sqrt{\text{means2d}^2 - \text{temp}}$.

6. Radius and Validity:

- $\text{radius} = \text{ceil}(3 \cdot \max(\text{extents}))$. Apply depth and image bound checks.

3 Computation of AABB, Means, and Extents

The AABB is $[\text{means2d}[0] \pm \text{radius}, \text{means2d}[1] \pm \text{radius}]$, with means2d and extents derived as above. This ensures tight bounds without T_{sl} inversion, using dot products for efficiency. The code implements these via summations, linking directly to the conic elements.

While the paper employs ray-splat intersection for rendering, bounding uses this conic approach with affine approximation.